

GUJARAT TECHNOLOGICAL UNIVERSITY**BE - SEMESTER-VI EXAMINATION – SUMMER 2025****Subject Code: 3160713****Date: 04-06-2025****Subject Name: Web Programming****Time: 10:30 AM TO 01:00 PM****Total Marks:70****Instructions:**

1. Attempt all questions.
2. Make suitable assumptions wherever necessary.
3. Figures to the right indicate full marks.
4. Simple and non-programmable scientific calculators are allowed.

		MARKS
Q.1	(a) Differentiate GET and POST method.	03
	(b) What is HTTP? Explain HTTP Request and Response.	04
	(c) Explain the points which should be considered for planning a website and also different design issue at the time of designing an effective website.	07
Q.2	(a) What is navigation? List the characteristics of effective navigation.	03
	(b) Explain following html tags with proper example. 1. 2. <frameset>	04
	(c) Explain different ways to write the CSS with example.	07
	OR	
	(c) What is HTML form tag? Explain the different form attributes and design a simple form to register student including name, roll no., semester, email, mobile number, gender, hobbies, and feedback, and also create two buttons submit and reset.	07
Q.3	(a) Explain pop-up boxes in Java-Script with example.	03
	(b) Write Java Script to validate Mobile Number. 1) Entered value must be numeric 2) 10 digit length	04
	(c) What is JavaScript event handling? List the major events and show use of at least one event by writing JavaScript code.	07
	OR	
Q.3	(a) Explain Session & Cookie in PHP.	03
	(b) Write a PHP script to print following pattern: 1 1 0 1 0 1 0 1 0 1 1 0 1 0 1	04
	(c) What is PHP Form processing? Write an HTML and PHP program which accepts N as input and print N Fibonacci numbers as list.	07
Q.4	(a) What is use of “break” and “continue” statements in PHP scripts?	03
	(b) Write an HTML and JavaScript program which accepts N as input and print N is odd or even number.	04
	(c) What is document object model? Explain any two objects with example.	07
	OR	
Q.4	(a) Differentiate synchronous and asynchronous web programming.	03
	(b) What are web services? Explain role of SOAP in Web services.	04
	(c) Write a PHP script to open, close, read and write into a file.	07

- Q.5** (a) Explain callbacks in JavaScript. **03**
(b) Explain the use of following arrays in PHP. **04**
1) \$_REQUEST [] 2) \$_SERVER []
(c) What is PHP? How can you connect to database in PHP? Show the simple database operation using PHP with proper example. **07**

OR

- Q.5** (a) Explain Regular Expression in PHP. **03**
(b) What is JQuery? What are the advantages of JQuery? **04**
(c) Write HTML and PHP program to read and store book information such as book id, title, author and price from database table. Output the data to a webpage in tabular format. **07**

Q.1 (a) Differentiate GET and POST method.

S.No	Feature	GET Method	POST Method
1	Purpose	Used to request data from the server.	Used to send data to the server for processing.
2	Data Visibility	Data is appended to the URL and visible in the browser's address bar.	Data is sent in the request body and not visible in the URL.
3	Data Length Limit	Limited (depends on browser and server, usually ~2000 characters).	No practical limit; can send large amounts of data.
4	Caching	Requests can be cached by browser or proxy.	Requests are not cached by default.
5	Bookmarking	URL with parameters can be bookmarked.	Cannot bookmark request with body data.
6	Security	Less secure; sensitive data appears in URL.	More secure than GET; data is not exposed in URL.
7	Use Case	Retrieving/searching data (e.g., search forms, queries).	Submitting forms, uploading files, sending sensitive data.
8	Idempotency	Safe and idempotent; multiple requests produce the same result.	Not idempotent; multiple requests may cause changes on server (e.g., insert).

(b) What is HTTP? Explain HTTP Request and Response.

HTTP (HyperText Transfer Protocol)

Definition:

HTTP is an **application layer protocol** used for **communication between a client (usually a web browser) and a server** over the internet. It defines **how messages are formatted and transmitted**, and how web servers and browsers should respond to various commands.

- HTTP works on the **request-response model**.
- It is **stateless**, meaning each request is independent and the server does not remember previous requests.
- It typically uses **TCP port 80** (HTTP) and **port 443** (HTTPS for secure communication).

HTTP Request

Definition:

An HTTP request is a **message sent by the client to the server** asking for a resource or requesting the server to perform an action.

Components of HTTP Request:

1. **Request Line** – Contains:
 - **Method** (GET, POST, PUT, DELETE, etc.)
 - **URL** of the resource
 - **HTTP version**Example:
2. GET /index.html HTTP/1.1
 3. **Request Headers** – Provide additional information about the request, such as:
 - Host: www.example.com
 - User-Agent: Mozilla/5.0
 - Content-Type: application/json
 4. **Request Body (Optional)** – Contains data sent to the server (mainly in POST or PUT requests).
-

HTTP Response

Definition:

An HTTP response is a **message sent by the server back to the client** after processing the request. It contains the requested resource or status information.

Components of HTTP Response:

1. **Status Line** – Contains:
 - **HTTP version**
 - **Status code** (200, 404, 500, etc.)
 - **Reason phrase**Example:
2. HTTP/1.1 200 OK
 3. **Response Headers** – Provide information about the server and the data:
 - Content-Type: text/html
 - Content-Length: 1024
 - Set-Cookie: sessionId=12345
 4. **Response Body (Optional)** – Contains the actual content (HTML, JSON, image, etc.) that the client requested.
-

(c) Explain the points which should be considered for planning a website and also different design issue at the time of designing an effective website.

Planning a Website:

Before starting website development, **careful planning** ensures the website achieves its goals efficiently. Below are the key considerations:

1. Purpose and Objectives

- Determine why the website is being created. Is it for **business promotion, online sales, education, portfolio, or blogging**?
- Clearly define the **goals**: increasing sales, providing information, attracting traffic, or offering services online.
- Establish measurable objectives to track website success, e.g., number of visitors, sales conversions, or newsletter sign-ups.

2. Target Audience Analysis

- Identify **who will use the website**: age group, profession, technical knowledge, interests, and region.
- Analyze user preferences to decide design, content style, and features.
- Understanding the audience helps in **designing navigation and content** that suits their needs.

3. Content Planning

- Plan the **type and volume of content**: text, images, videos, audio, downloadable files.
- Create a **content hierarchy**: main pages (Home, About, Services), subpages, and links between pages.
- Ensure content is **relevant, engaging, accurate, and updated regularly**.

4. Website Structure and Navigation

- Create a **site map** outlining main sections and their relationship.
- Decide how users will navigate: horizontal menu, sidebar menu, dropdowns, footer links.
- Plan for **user-friendly navigation**: users should reach any page in **3 clicks or fewer**.

5. Technology and Platform

- Choose appropriate **development technologies**: HTML/CSS/JS for static sites, CMS like WordPress for content-heavy sites, or frameworks like React/Vue for dynamic apps.
- Decide on **server-side technologies** if backend functionality is required: PHP, Node.js, Python, or databases like MySQL/MongoDB.
- Consider **cross-browser compatibility** and **mobile responsiveness**.

6. Budget and Resources

- Calculate **total cost** for development, domain registration, hosting, and maintenance.
- Identify **human resources**: developers, designers, content writers, SEO specialists.
- Assess **time resources** and plan for delays in content creation or testing.

7. Timeline and Milestones

- Divide the project into **phases**: planning, design, development, testing, deployment.
- Set **realistic deadlines** for each phase.
- Include **time for revisions, debugging, and client approvals**.

8. Security and Legal Considerations

- Plan for **data security**: use HTTPS, secure forms, encrypted storage for sensitive data.
- Implement **user authentication** if needed.
- Ensure compliance with **privacy laws and regulations** (like GDPR for European users).

9. SEO and Accessibility

- Incorporate **search engine optimization** (SEO) from the planning stage.
- Use **semantic HTML**, meta tags, proper heading hierarchy, and keywords in content.
- Ensure **accessibility** for users with disabilities (screen readers, color contrast, keyboard navigation).

10. Maintenance Plan

- Plan for **regular updates**: content refresh, security patches, software updates.
- Monitor **analytics** to track performance and make improvements.

Design Issues in Creating an Effective Website

Even with good planning, poor design can ruin user experience. Consider these design issues carefully:

1. Website Layout

- Layout should be **clean, organized, and visually appealing**.
- Use grids and alignment to ensure content is structured logically.
- Avoid overcrowding the page with too many elements.

2. Navigation Design

- Menus should be **intuitive and simple**: users should not get lost.
- Include **breadcrumbs** to show users their current location.
- Use **consistent navigation** across all pages.

3. Color Scheme and Typography

- Use **colors that reflect the brand** and are easy on the eyes.
- Ensure sufficient **contrast** between text and background.
- Choose fonts that are **readable on all devices** and consistent in style.

4. Loading Speed and Performance

- Optimize images and videos to reduce **page load times**.
- Minimize use of heavy scripts or unnecessary plugins.

- Fast loading increases user satisfaction and **SEO ranking**.

5. Mobile Responsiveness

- Design should **adapt to all screen sizes** (desktop, tablet, mobile).
- Use responsive frameworks like **Bootstrap or CSS media queries**.
- Test on multiple devices to ensure proper layout and usability.

6. Consistency

- Keep design elements **uniform**: buttons, headers, footers, colors, and spacing.
- Consistency creates a **professional and trustworthy look**.

7. Content Readability

- Use **short paragraphs, headings, bullet points, and whitespace**.
- Avoid walls of text; make content **scannable** for users.
- Use **images and multimedia** to complement text and make it engaging.

8. User Experience (UX)

- Forms, buttons, and interactive elements should be **easy to use**.
- Reduce unnecessary steps to complete actions.
- Ensure feedback is provided on actions (e.g., form submission confirmation).

9. SEO and Accessibility in Design

- Use **semantic HTML tags**, alt text for images, and proper heading hierarchy.
- Optimize URLs, meta tags, and links for search engines.
- Ensure accessibility for users with **visual, hearing, or motor impairments**.

10. Security in Design

- Protect forms and data entry points against **SQL injection, XSS, and other attacks**.
- Include secure login, CAPTCHA for bots, and proper session management.

Q.2 (a) What is navigation? List the characteristics of effective navigation.

Definition:

Navigation in web design refers to the **system or mechanism that allows users to move between different pages or sections of a website**.

It helps users **find information quickly and easily**, guiding them through the website's content. Navigation can include menus, links, buttons, search bars, breadcrumbs, and other interactive elements.

Characteristics of Effective Navigation:

1. **Clarity and Simplicity**
 - Navigation should be **easy to understand** at first glance.
 - Menu items should use **simple and meaningful labels** (e.g., “About Us,” “Contact,” “Services”).
2. **Consistency**
 - Navigation should remain **consistent across all pages**.
 - Users should not have to re-learn navigation on each page.
3. **Predictability**
 - Users should be able to **predict where a link will take them** based on its label.
 - Avoid misleading labels or unexpected navigation behaviors.
4. **Visibility and Accessibility**
 - Navigation menus should be **visible and easy to find** on all pages.
 - Use contrasting colors, readable fonts, and accessible placement for all users.
5. **Efficiency**
 - Users should be able to **reach any page within a few clicks** (ideally 3 or fewer).
 - Include shortcuts or quick links for important sections.
6. **Feedback**
 - The navigation system should **indicate the current page** (e.g., highlighted menu item).
 - Provide feedback when links are clicked or hovered.
7. **Scalability**
 - Navigation should **adapt as the website grows** with more pages and content.
 - Dropdowns or multi-level menus can be used for large sites.
8. **Accessibility**
 - Navigation should be **usable for all users**, including those with disabilities.
 - Keyboard navigation, screen reader support, and proper HTML structure are essential.
9. **Responsive Design**
 - Navigation should **work well on all devices** (desktop, tablet, mobile).
 - Use collapsible menus or hamburger menus for smaller screens.
10. **SEO-Friendly**
 - Proper navigation structure helps **search engines index pages**.
 - Use descriptive link texts and avoid broken links.

(b) Explain following html tags with proper example. 1. 2.<frameset>

1. Tag

Definition:

The tag is used to **embed images in an HTML page**.

- It is an **empty tag**, meaning it does not have a closing tag.

- The tag requires a **source attribute (src)** to specify the image file.

Attributes of Tag:

Attribute	Description
src	Specifies the path or URL of the image.
alt	Provides alternative text if the image cannot be displayed. Important for accessibility.
width	Sets the width of the image in pixels or percentage.
height	Sets the height of the image in pixels or percentage.
title	Provides additional information about the image on hover.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Image Example</title>
</head>
<body>
  <h2>HTML Image Example</h2>
  
</body>
</html>
```

2. <frameset> Tag

Definition:

The <frameset> tag is used to **divide a browser window into multiple frames**, where each frame can display a different HTML document.

Note: <frameset> is **deprecated in HTML5**, but still used in older HTML versions. Modern alternatives include **CSS layout with <i>iframe</i>** or **flex/grid layout**.

Attributes of <frameset> Tag:

Attribute	Description
rows	Specifies how many horizontal rows to divide the window into.
cols	Specifies how many vertical columns to divide the window into.
border	Sets the thickness of the frame border.

- Each frame is defined using the `<frame>` tag inside the `<frameset>`.
- The `<frame>` tag has attributes like `src` (page to display), `name`, `scrolling`, etc.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Frameset Example</title>
</head>
<frameset rows="50%,50%">
  <frame src="header.html" name="topFrame">
  <frame src="content.html" name="bottomFrame">
</frameset>
</html>
```

(c) Explain different ways to write the CSS with example.

Different Ways to Write CSS (Cascading Style Sheets)

CSS (Cascading Style Sheets) is a language used to **control the appearance and layout of HTML elements**. Using CSS, we can define **colors, fonts, spacing, positioning, and other visual aspects** of a website.

CSS separates **content from presentation**, which makes websites easier to maintain and more flexible. There are **three main ways** to apply CSS in web design:

1. Inline CSS

Definition:

- Inline CSS is written **directly inside an HTML tag** using the `style` attribute.
- It affects **only that particular element**, so it is not reusable.

Syntax:

```
<tagname style="property:value;">
```

Characteristics:

- **Highest priority** in the CSS cascade (overrides internal or external CSS if conflicts exist).
- **Quick to implement** for small changes or testing.
- Not suitable for large websites because it **mixes content with style**, making code harder to maintain.

Example :

```
<h1 style="color: blue; text-align: center;">Welcome to Inline CSS</h1>
```

Explanation:

- The <h1> text is **blue and centered**.
 - Inline CSS is useful for **specific, one-time styling**, like highlighting a single element.
-

2. Internal CSS (Embedded CSS)

Definition:

- Internal CSS is written **inside the <style> tag in the <head> section** of the HTML page.
- It applies **to all elements of the page** that match the specified selectors.

Syntax:

```
<style>
selector {
    property: value;
}
</style>
```

Characteristics:

- **Applies to the entire page**, not just a single element.
- Useful for **single-page websites** or unique page-specific styling.
- Keeps styles separate from individual elements, making HTML slightly cleaner than inline CSS.
- Cannot be reused for other pages, so less efficient for multi-page websites.

Example:

```
<style>
p { color: green; font-size: 16px; }
</style>
<p>This paragraph uses internal CSS.</p>
```

Explanation:

- All <p> tags on that page will have **green text with font size 16px**.
 - Allows styling multiple elements without repeating the `style` attribute.
-

3. External CSS

Definition:

- External CSS is written in a **separate .css file** and linked to HTML pages using the `<link>` tag.
- It allows **consistent styling across multiple pages**, making it ideal for large websites.

Syntax:

```
<link rel="stylesheet" href="style.css">
```

Characteristics:

- **Reusable** across multiple pages, reducing redundancy.
- Keeps HTML code **clean and readable**.
- Easy to **update** the look of the entire website by modifying a single CSS file.
- Preferred method for **professional web development**.

Example :

```
<link rel="stylesheet" href="style.css">
h1 { color: red; text-align: center; }
```

Explanation:

- All `<h1>` elements in pages linked to `style.css` will appear **red and centered**.
- External CSS is **most maintainable and scalable** for large websites.

OR (c) What is HTML form tag? Explain the different form attributes and design a simple form to register student including name, roll no., semester, email, mobile number, gender, hobbies, and feedback, and also create two buttons submit and reset.

HTML Form Tag

Definition:

The `<form>` tag in HTML is used to **create interactive forms on web pages** that allow users to **input data and submit it to a server** for processing. Forms are essential for **registration, login, feedback, surveys, and other data collection tasks**.

A form can include **text boxes, radio buttons, checkboxes, dropdowns, buttons, and more**.

Attributes of `<form>` Tag

Attribute	Description	Example
action	Specifies the URL of the server-side script that will process the form data.	<code><form action="submit.php"></code>
method	Defines the HTTP method used to send form data. Values: GET or POST.	<code><form method="POST"></code>
name	Assigns a name to the form . Useful in JavaScript to refer to the form.	<code><form name="studentForm"></code>
id	Provides a unique identifier for the form, useful for CSS or JS targeting.	<code><form id="regForm"></code>
target	Specifies where to display the response after form submission. Options: <code>_self</code> , <code>_blank</code> , <code>_parent</code> , <code>_top</code> .	<code><form target="_blank"></code>
enctype	Specifies the encoding type for form submission. Common values:	

HTML Form Example: Student Registration Form

```
<form action="submit.php" method="POST">
  <!-- Name -->
  Name: <input type="text" name="name" required><br><br>

  <!-- Roll Number -->
  Roll No.: <input type="text" name="rollno" required><br><br>

  <!-- Semester -->
  Semester:
  <select name="semester" required>
    <option value="">Select</option>
    <option value="1">1</option>
    <option value="2">2</option>
    <option value="3">3</option>
    <option value="4">4</option>
  </select><br><br>

  <!-- Email -->
  Email: <input type="email" name="email" required><br><br>

  <!-- Mobile Number -->
  Mobile: <input type="tel" name="mobile" pattern="[0-9]{10}"
required><br><br>

  <!-- Gender -->
  Gender:
  <input type="radio" name="gender" value="Male" required> Male
  <input type="radio" name="gender" value="Female" required> Female<br><br>

  <!-- Hobbies -->
  Hobbies:
  <input type="checkbox" name="hobbies" value="Reading"> Reading
  <input type="checkbox" name="hobbies" value="Sports"> Sports
  <input type="checkbox" name="hobbies" value="Music"> Music<br><br>

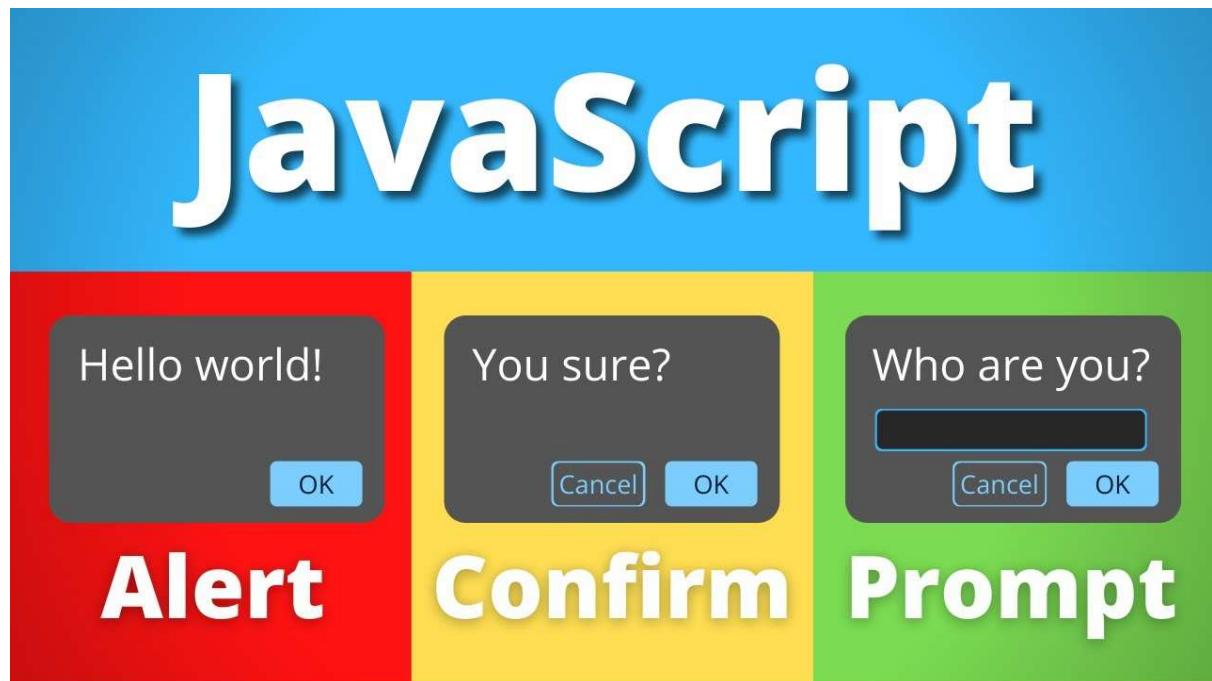
  <!-- Feedback -->
```

```
Feedback:<br>
<textarea name="feedback" rows="4" cols="40"></textarea><br><br>

<!-- Buttons -->
<input type="submit" value="Submit">
<input type="reset" value="Reset">
</form>
```

Q.3 (a) Explain pop-up boxes in JavaScript with example.

Pop-Up Boxes in JavaScript



Pop-Up Boxes in JavaScript

Definition:

Pop-up boxes in JavaScript are **dialog boxes that appear on a webpage** to communicate with users. They are used to **display messages, take input, or confirm actions**. Pop-ups are **modal**, meaning the user must respond before interacting with the rest of the webpage.

There are **three types of pop-ups: Alert Box, Confirm Box, Prompt Box**.

1. Alert Box

Explanation:

- Used to **display a message** to the user.
- Contains only an **OK button**.

- Pauses execution until the user clicks OK.

Example:

```
alert("Welcome to our website!");
```

Features:

- Simple **informational messages**.
 - **No user input** required.
 - Useful for **warnings or notifications**.
-

2. Confirm Box

Explanation:

- Used to **ask for user confirmation**.
- Contains **OK and Cancel buttons**.
- Returns a **boolean value** depending on the user's choice.

Example:

```
var result = confirm("Are you sure you want to delete?");
```

Features:

- Suitable for **critical actions** like deletion or submission.
 - Execution is **paused until response**.
 - Returns **true (OK) or false (Cancel)**.
-

3. Prompt Box

Explanation:

- Used to **take input from the user** via a text field.
- Contains **OK and Cancel buttons**.
- Returns the **user-entered value** or **null** if canceled.

Example:

```
var name = prompt("Enter your name:", "Guest");
```

Features:

- Collects **dynamic user input**.
- Can be used for **personalized messages**.

- Execution is **paused until input is submitted or canceled**.

(b) Write Java Script to validate Mobile Number. 1) Entered value must be numeric 2) 10 digit length

Requirement:

1. Entered value **must be numeric**
 2. Mobile number **must be exactly 10 digits**
-

JavaScript Code

```
<input type="text" id="mobile">
<button onclick="validateMobile()">Submit</button>

<script>
function validateMobile() {
    var mobile = document.getElementById("mobile").value;

    if (isNaN(mobile)) {
        alert("Mobile number must contain only digits");
        return false;
    }

    if (mobile.length != 10) {
        alert("Mobile number must be 10 digits");
        return false;
    }

    alert("Valid mobile number");
}
</script>
```

OUTPUT:

Input: 98ab56@123

Output:

→ A pop-up box appears showing:

**"Mobile number must contain
only digits"**

(c) What is JavaScript event handling? List the major events and show use of at least one event by writing JavaScript code.

JavaScript event handling is a programming technique used to make web pages **interactive and responsive** by allowing them to react to **user actions and browser-generated events**.

An **event** is any action or occurrence detected by the web browser, such as clicking a button, typing on the keyboard, moving the mouse, loading a page, submitting a form, or resizing the browser window.

Event handling refers to the process of **capturing these events and executing specific JavaScript code in response to them**. When an event occurs, the browser automatically triggers the associated **event handler**, which is a JavaScript function written by the developer to perform a particular task.

Without event handling, a website would be **static**, meaning it would only display information and would not respond to user interactions.

Components of Event Handling:

1. **Event Source** – The HTML element on which the event occurs (button, textbox, form, window, etc.)
2. **Event Type** – The type of action performed (click, keypress, load, submit, etc.)
3. **Event Handler** – The JavaScript code or function that executes when the event occurs

Importance of JavaScript Event Handling:

- Enables **interaction between user and webpage**
- Used for **form validation and data checking**
- Allows **dynamic content updates** without page reload
- Improves **usability and user experience**
- Forms the foundation of **modern web applications**
- Helps in handling **real-time user actions**

Major JavaScript Events:

1. Mouse Events

Mouse events occur when the user interacts with the mouse.

- `onclick` – Fired when an element is clicked
- `ondblclick` – Fired on double-click
- `onmouseover` – Fired when mouse enters an element

- `onmouseout` – Fired when mouse leaves an element
-

2. Keyboard Events

Keyboard events occur when keys are pressed or released.

- `onkeydown` – Key is pressed down
 - `onkeyup` – Key is released
 - `onkeypress` – Key is pressed and released
-

3. Form Events

Form events occur when user interacts with form elements.

- `onsubmit` – When form is submitted
 - `onreset` – When form is reset
 - `onchange` – When input value changes
 - `onfocus` – When input field gains focus
 - `onblur` – When input field loses focus
-

4. Window / Document Events

These events relate to browser window activities.

- `onload` – Page is fully loaded
 - `onresize` – Browser window is resized
 - `onunload` – Page is closed or refreshed
-

[Use of JavaScript Event Handling \(Example – onclick Event\)](#)

Explanation:

The `onclick` event is triggered when the user **clicks on an HTML element**, such as a button. This event is commonly used to perform actions like displaying messages, validating input, or executing logic based on user interaction.

Short JavaScript Code Example:

```
<button onclick="displayMessage()">Click Here</button>
```

```
<script>
function displayMessage() {
    alert("Welcome! Button was clicked.");
}
</script>
```

Working of the Program:

1. A button is displayed on the webpage
 2. User clicks the button
 3. The `onclick` event is triggered
 4. JavaScript function `displayMessage()` is executed
 5. A pop-up message is shown to the user
-

Advantages of Event Handling:

- Makes web pages **interactive**
- Reduces unnecessary page reloads
- Improves **user engagement**
- Enables **real-time response** to actions
- Simplifies **client-side processing**

OR

Q.3 (a) Explain Session & Cookie in PHP.

1. Session in PHP

A **session** in PHP is a way to **store user-specific data on the server** so that it can be used across multiple web pages. Since HTTP is a **stateless protocol**, it does not remember user information between requests. Sessions are used to **maintain state**, such as login details, user preferences, and shopping cart information.

In a session, PHP generates a **unique session ID** for each user. This session ID is usually stored on the client side as a **cookie**, and the actual data is stored on the **server**.

Working of Session:

1. User requests a web page
2. Server creates a **session ID**
3. Session data is stored on the **server**
4. Session ID is sent to the browser

5. Browser sends session ID with every request
 6. Server identifies the user using session ID
-

Features of PHP Session:

- Data is stored on the **server**
 - More **secure** than cookies
 - Can store **large amounts of data**
 - Used for **authentication and authorization**
 - Data is destroyed when session expires or browser closes
-

Short PHP Example (Session):

```
session_start();  
$_SESSION["username"] = "nian";
```

2. Cookie in PHP

Long Definition:

A **cookie** is a small piece of data stored on the **client's browser** by the web server. Cookies are used to **remember information about the user**, such as login name, theme preference, or language selection, so that the data is available even after the browser is closed.

Each cookie consists of a **name, value, expiration time, and path**. Cookies are automatically sent by the browser to the server with every request to the same domain.

Working of Cookie:

1. Server sends cookie to the browser
 2. Browser stores the cookie locally
 3. Browser sends cookie back to server on next request
 4. Server reads cookie data
-

Features of PHP Cookie:

- Data is stored on the **client side**
- Less secure compared to sessions
- Can store **limited data size**
- Can persist for **days, months, or years**
- Used for **user preferences and tracking**

Short PHP Example (Cookie):

```
setcookie("username", "nian", time()+3600);
```

(b) Write a PHP script to print following pattern:

1

1 0

1 0 1

0 1 0 1

1 0 1 0 1 0

```
<?php
$rows = 5;

for ($i = 1; $i <= $rows; $i++) {

    for ($j = 1; $j <= $i; $j++) {

        if ($j % 2 == 1) {
            echo "1 ";
        } else {
            echo "0 ";
        }

    }

    echo "<br>";
}
?>
```

(c) What is PHP Form processing? Write an HTML and PHP program which accepts N as input and print N Fibonacci numbers as list.

PHP form processing is the technique used to **collect, process, and respond to data submitted by a user through an HTML form** using PHP scripts. Forms are the primary way through which users interact with a website, such as entering numbers, names, emails, passwords, or feedback.

When a user submits a form, the data is sent to the server using HTTP methods like **GET** or **POST**. PHP then retrieves this data, processes it according to program logic, and generates a dynamic response.

Why PHP Form Processing is Needed:

- To accept **user input**
 - To perform **calculations or logic**
 - To validate user data
 - To store or display data
 - To generate **dynamic web pages**
-

Steps in PHP Form Processing:

1. User fills the HTML form
2. Form data is submitted to the server
3. PHP script receives the data
4. PHP processes the data
5. Output is displayed to the user

Program to Accept N and Print N Fibonacci Numbers

HTML Form (Input N):

```
<form method="post">
  Enter value of N:
  <input type="text" name="num">
  <input type="submit" name="submit" value="Generate">
</form>
```

PHP Code (Processing & Output):

```
<?php
if (isset($_POST["submit"])) {
    $n = $_POST["num"];

    $a = 0;
```

```
$b = 1;

echo "<ul>";

for ($i = 1; $i <= $n; $i++) {
    echo "<li>$a</li>";
    $c = $a + $b;
    $a = $b;
    $b = $c;
}

echo "</ul>";
}
?>
```

OUTPUT:

- 0
- 1
- 1
- 2
- 3
- 5

Q.4 (a) What is use of “break” and “continue” statements in PHP scripts?

1. break Statement in PHP

Definition:

The **break** statement is used to **immediately terminate the execution of a loop or switch statement**. When **break** is encountered, the control exits from the loop and moves to the **next statement after the loop**.

Use of break:

- To **stop loop execution** when a specific condition is met
 - To exit from **nested loops**
 - Commonly used in **search operations**
 - Used to terminate a **switch case**
-

Working of `break`:

- PHP checks the condition inside the loop
 - If the condition becomes true, `break` is executed
 - Loop execution stops instantly
 - Control shifts outside the loop
-

Example (`break`):

```
for ($i = 1; $i <= 5; $i++) {  
    if ($i == 3) {  
        break;  
    }  
    echo $i;  
}
```

Output:

12

2. `continue` Statement in PHP

Definition:

The **`continue` statement** is used to **skip the current iteration of a loop** and immediately move to the **next iteration**, without terminating the entire loop.

Use of `continue`:

- To **skip unwanted values**
 - To avoid execution of certain code for specific conditions
 - Useful in **filtering logic**
 - Helps in **clean and readable code**
-

Working of `continue`:

- PHP evaluates the loop condition
 - When `continue` is executed:
 - Remaining statements in the current loop iteration are skipped
 - Loop proceeds with the next iteration
-

Example (continue):

```
for ($i = 1; $i <= 5; $i++) {  
    if ($i == 3) {  
        continue;  
    }  
    echo $i;  
}
```

Output:

1245

(b) Write an HTML and JavaScript program which accepts N as input and print N is odd or even number.

```
<!DOCTYPE html>  
  
<html>  
  
<head>  
    <title>Odd or Even</title>  
</head>  
  
<body>  
  
    Enter a number:  
  
    <input type="text" id="num">  
  
    <button onclick="checkNumber()">Check</button>  
  
  
    <script>  
  
    function checkNumber() {  
  
        var n = document.getElementById("num").value;  
  
  
        if (n % 2 == 0) {  
  
            alert("The number is Even");  

```

```
    } else {  
        alert("The number is Odd");  
    }  
}  
  
</script>  
</body>  
</html>
```

Output:

```
Input: 10  
Alert: The number is Even  
Input: 7  
Alert: The number is Odd
```

(c) What is document object model? Explain any two objects with example.

The **Document Object Model (DOM)** is a **programming interface** that represents an HTML or XML document as a **tree-like structure of objects**. In the DOM, every part of a web page—such as elements, attributes, text, and even the document itself—is treated as an **object**.

When a web page is loaded, the browser creates the DOM of that page. Using **JavaScript**, programmers can access, modify, add, or delete HTML elements dynamically through the DOM. This allows web pages to become **interactive and dynamic** without reloading the page.

In simple words, **DOM acts as a bridge between HTML and JavaScript**, enabling JavaScript to control and manipulate the content, structure, and style of a webpage.

Key Features of DOM:

- Represents webpage as a **tree of objects**
- Allows **dynamic modification** of HTML
- Language-independent (commonly used with JavaScript)
- Supports **event handling**

- Enables **client-side interactivity**
-

DOM Tree Structure (Concept):

- **Document** (root)
 - **HTML**
 - **Head**
 - **Body**
 - Elements like `div`, `p`, `input`, `button`, etc.
-

DOM Objects

There are many objects in DOM such as **document**, **window**, **form**, **element**, **input**, **button**, etc.

Below are **any two DOM objects** explained with examples.

1. Document Object

Definition:

The **document object** is the **root object of the DOM**. It represents the entire HTML document loaded in the browser. All other DOM objects are accessed through the document object.

It is mainly used to:

- Access HTML elements
 - Change content and structure
 - Handle page-level operations
-

Uses of Document Object:

- Access elements using ID, name, tag, or class
 - Write content to the page
 - Change HTML dynamically
-

Example (Document Object):

```
<p id="msg">Hello</p>
```

```
<script>
document.getElementById("msg").innerHTML = "Welcome to DOM";
</script>
```

Explanation:

- `document.getElementById()` accesses the HTML element
 - `innerHTML` changes its content dynamically
-

2. Form Object

Definition:

The **form object** represents an HTML `<form>` element in the DOM. It allows JavaScript to **access form fields, validate input, and control form submission**.

Each form in a webpage becomes a form object that can be accessed using the document object.

Uses of Form Object:

- Access input fields
 - Validate user input
 - Control form submission
 - Read and modify form values
-

Example (Form Object):

```
<form name="myForm">
  <input type="text" name="uname">
</form>

<script>
document.myForm.uname.value = "Komal";
</script>
```

Explanation:

- `document.myForm` accesses the form
 - `uname.value` sets value of input field
-

OR

Q.4 (a) Differentiate synchronous and asynchronous web programming.

No.	Synchronous Web Programming	Asynchronous Web Programming
1	Operations are executed one after another	Operations are executed independently
2	Each request waits for the previous one to complete	Requests do not wait for previous tasks
3	Program execution is blocked until task finishes	Program execution is non-blocking
4	User interface may freeze or become unresponsive	User interface remains responsive
5	Slower performance for time-consuming tasks	Faster performance for long-running tasks
6	Simple to understand and debug	Slightly complex due to callbacks/promises
7	Suitable for small or simple applications	Suitable for modern, large-scale applications
8	Example: Traditional page reload after form submit	Example: AJAX, Fetch API, async JavaScript

(b) What are web services? Explain role of SOAP in Web services.

Web services are software systems designed to support communication and data exchange between different applications over the Internet. They allow applications developed in **different programming languages**, running on **different platforms and operating systems**, to interact with each other in a standardized way.

Web services use **open standards** such as HTTP, XML, SOAP, and WSDL to enable **interoperability** between heterogeneous systems.

Characteristics of Web Services:

- Platform independent
- Language independent
- Loosely coupled
- Interoperable
- Use standard web protocols
- Enable machine-to-machine communication

Types of Web Services:

- **SOAP-based Web Services**
 - **RESTful Web Services**
-

Examples of Web Services:

- Online payment gateways
 - Weather information services
 - Online banking systems
 - Travel booking APIs
-

[SOAP \(Simple Object Access Protocol\)](#)

Definition:

SOAP stands for **Simple Object Access Protocol**. It is a **protocol used for exchanging structured information in web services**. SOAP defines a **standard message format** that allows applications to communicate reliably over a network.

SOAP messages are written in **XML format** and are typically transmitted using **HTTP or HTTPS**.

[Role of SOAP in Web Services](#)

SOAP plays a **central role** in enabling communication between web service providers and consumers.

1. Message Formatting

SOAP defines a **standard XML structure** for request and response messages. This ensures that data exchanged between applications is **understood uniformly**, regardless of the programming language or platform.

2. Communication Protocol

SOAP acts as a **communication protocol** between client and server. It enables **remote procedure calls (RPC)**, allowing a client to invoke methods on a remote server.

3. Platform and Language Independence

Since SOAP uses **XML**, it allows web services built using Java, PHP, .NET, Python, etc., to communicate seamlessly.

4. Error Handling

SOAP provides built-in **error handling** using **SOAP Faults**, which return standardized error messages when a request fails.

5. Security Support

SOAP supports advanced security features such as:

- Authentication
- Encryption
- Digital signatures

This makes SOAP suitable for **enterprise-level and financial applications**.

6. Reliability and Transaction Support

SOAP supports **reliable messaging and transactional integrity**, ensuring messages are delivered correctly and in order.

Structure of SOAP Message :

A SOAP message consists of:

1. **Envelope** – Defines the start and end of the message
 2. **Header** – Contains optional metadata
 3. **Body** – Contains actual request or response data
 4. **Fault** – Contains error information
-

Advantages of SOAP in Web Services:

- Highly secure
- Standardized messaging format
- Supports complex operations

- Reliable communication

(c) Write a PHP script to open, close, read and write into a file.

```
<?php
// Open file in write mode
$file = fopen("data.txt", "w");

// Write data into file
fwrite($file, "Welcome to PHP File Handling.\n");
fwrite($file, "This file contains sample text.");

// Close the file
fclose($file);

// Open file in read mode
$file = fopen("data.txt", "r");

// Read data from file
$content = fread($file, filesize("data.txt"));
echo $content;

// Close the file
fclose($file);
?>
```

Q.5 (a) Explain callbacks in JavaScript.

A **callback function** is a function that is not executed immediately but is **called back** at a later point in time when a specific operation has finished. JavaScript uses callbacks to ensure that certain code runs **only after a particular task is completed**, especially in situations where tasks take time and should not block program execution.

Callbacks help JavaScript remain **non-blocking and efficient**, allowing other code to run while waiting for time-consuming operations to complete.

Why Callbacks are Needed:

JavaScript is **single-threaded**, meaning it executes one task at a time. Without callbacks, long-running tasks would block execution and freeze the webpage. Callbacks allow JavaScript to:

- Handle asynchronous tasks
 - Execute code in the correct order
 - Improve responsiveness
 - Avoid blocking the main thread
-

How Callbacks Work :

1. A function is defined
 2. Another function is passed as a parameter
 3. When the main task finishes, the callback function is executed
 4. The result is then processed
-

Types of Callbacks:

1. Synchronous Callbacks

Executed immediately during program execution.

```
function greet(name, callback) {  
    callback();  
}
```

2. Asynchronous Callbacks

Executed after a task completes, such as a timer or server response.

```
setTimeout(function() {  
    console.log("Executed later");  
}, 1000);
```

Features of Callbacks:

- Enable **asynchronous programming**
 - Improve **program flow control**
 - Help in handling **events**
 - Widely used in **AJAX and APIs**
 - Allow execution after task completion
-

Advantages of Callbacks:

- Non-blocking execution
 - Efficient handling of delayed tasks
 - Better control over execution order
-

Disadvantages of Callbacks:

- Can lead to **callback hell** (nested callbacks)
- Code becomes **harder to read and maintain**
- Difficult error handling in complex cases

(b) Explain the use of following arrays in PHP.

1) `$_REQUEST [ ]` 2) `$_SERVER [ ]`

`$_REQUEST` is a **superglobal associative array** in PHP that is used to **collect data sent by the client to the server**. It contains the combined data of `$_GET`, `$_POST`, and `$_COOKIE` arrays.

This array allows programmers to **access form data without worrying about the request method** used.

Uses of `$_REQUEST`:

- To access form input values
- To retrieve data sent via GET or POST method
- To simplify form processing
- To access cookie values (if enabled)

Example (`$_REQUEST`):

```
$name = $_REQUEST["username"];  
echo $name;
```

2) \$_SERVER[] Array

Definition:

\$_SERVER is a **superglobal associative array** that contains **information about the server and the execution environment**. It provides details related to headers, paths, script locations, request methods, and server configuration.

This array is mainly used for **server-side and environment-related operations**.

Uses of \$_SERVER:

- To get server name and IP address
- To identify request method (GET or POST)
- To get script file path
- To access browser and client information
- To manage redirections and security checks

Example (\$_SERVER):

```
echo $_SERVER["SERVER_NAME"];
```

(c) What is PHP? How can you connect to database in PHP? Show the simple database operation using PHP with proper example.

What is PHP?

Definition:

PHP (Hypertext Preprocessor) is an **open-source, server-side scripting language** used to develop **dynamic and interactive web applications**. PHP code is executed on the **web server**, and the result is sent to the client's browser as **HTML**.

Features of PHP:

- Server-side scripting language

- Open source and free
 - Platform independent
 - Easy to learn and embed with HTML
 - Supports many databases (MySQL, PostgreSQL, Oracle, etc.)
 - Fast and secure for web development
-

Uses of PHP:

- Form handling
 - Database operations
 - Session and cookie management
 - Login and authentication systems
 - Dynamic website creation
-

How can you connect to a database in PHP?

PHP can connect to databases using two main methods:

1. **MySQLi (MySQL Improved)**
2. **PDO (PHP Data Objects)**

☞ In most basic and academic examples, **MySQLi** is commonly used.

Steps to connect database using PHP (MySQLi):

1. Define database connection parameters
 2. Create connection using `mysqli_connect()`
 3. Check connection status
 4. Perform database operations (INSERT, SELECT, UPDATE, DELETE)
 5. Close the connection
-

Database Connection in PHP (MySQLi)

Connection Syntax:

```
$conn = mysqli_connect("hostname", "username", "password",  
"database_name");
```

Simple Database Operation Using PHP (Example)

Example: Insert and Display Data from Database

Database Details:

- Database name: studentdb
- Table name: students

Table Structure:

id	name	age
----	------	-----

(A) HTML Form (input.html)

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Form</title>
</head>
<body>
  <form method="post" action="insert.php">
    Name: <input type="text" name="name" required><br><br>
    Age: <input type="number" name="age" required><br><br>
    <input type="submit" value="Save">
  </form>
</body>
</html>
```

(B) PHP Database Connection and Insert Operation (insert.php)

```
<?php
// Database connection
$conn = mysqli_connect("localhost", "root", "", "studentdb");

// Check connection
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}

// Get form data
$name = $_POST["name"];
$age = $_POST["age"];

// Insert query
$sql = "INSERT INTO students (name, age) VALUES ('$name', '$age')";

// Execute query
if (mysqli_query($conn, $sql)) {
    echo "Record inserted successfully";
} else {
    echo "Error: " . mysqli_error($conn);
}

// Close connection
mysqli_close($conn);
?>
```

(C) PHP Select (Read) Operation (display.php)

```
<?php
$conn = mysqli_connect("localhost", "root", "", "studentdb");

if (!$conn) {
    die("Connection failed");
}

$result = mysqli_query($conn, "SELECT * FROM students");

echo "<h3>Student Records</h3>";

while ($row = mysqli_fetch_assoc($result)) {
    echo "ID: " . $row["id"] . " ";
    echo "Name: " . $row["name"] . " ";
    echo "Age: " . $row["age"] . "<br>";
}

mysqli_close($conn);
?>
```

Output (Sample)

Record inserted successfully

Student Records

ID: 1 Name: Rahul Age: 20

ID: 2 Name: Neha Age: 22

Advantages of Using PHP with Database:

- Easy database connectivity
- Fast data processing
- Supports CRUD operations
- Secure with prepared statements
- Widely used in web applications

OR

Q.5 (a) Explain Regular Expression in PHP.

Regular Expression in PHP

A **Regular Expression (Regex)** is a sequence of characters that defines a **search pattern**.

In PHP, regular expressions are mainly used for **string matching, searching, and validation**. PHP supports **Perl Compatible Regular Expressions (PCRE)**, which are powerful and flexible.

Regular expressions help in validating user inputs such as **email IDs, mobile numbers, and passwords**.

They are also used to **find, replace, or extract data** from text. Regex improves accuracy and reduces manual string handling. In PHP, regex functions start with **preg_**.

Examples

1. Mobile Number Validation

```
$mobile = "9876543210";  
if (preg_match("/^[0-9]{10}$/", $mobile)) {  
    echo "Valid Mobile Number";  
} else {  
    echo "Invalid Mobile Number";  
}
```

2. Replace Word in String

```
$text = "Hello World";  
echo preg_replace("/World/", "PHP", $text);
```

Key Features of Regular Expression in PHP

- Used for **input validation**
- Powerful **pattern matching**
- Supports **search and replace**
- Works efficiently with strings
- Reduces code complexity
- Widely used in form processing

(b) What is JQuery? What are the advantages of JQuery?

jQuery is a **fast, lightweight, and open-source JavaScript library** designed to **simplify client-side scripting**. It makes tasks such as **HTML DOM manipulation, event handling, animations, and AJAX operations** much easier and faster. jQuery allows developers to write **less code with better readability**. It also provides **built-in support for cross-browser compatibility**, solving many JavaScript browser issues. jQuery follows the principle “**Write less, do more.**”

Key Features of jQuery

- Simple and powerful **DOM manipulation**
 - Easy **event handling**
 - Built-in **AJAX support**
 - Rich **animation and effects**
 - Cross-browser compatibility
 - Extensible through **plugins**
-

Advantages of jQuery

1. Easy to Learn and Use

jQuery syntax is simple and user-friendly, making it easier than pure JavaScript.

2. Less Code Required

Common tasks can be done with fewer lines of code, saving time and effort.

3. Cross-Browser Support

jQuery handles browser differences internally, ensuring consistent behavior.

4. Powerful DOM Manipulation

HTML elements can be easily selected, modified, added, or removed.

5. Event Handling Made Easy

Mouse, keyboard, and form events are handled using simple methods.

6. AJAX Support

Allows asynchronous data loading without refreshing the web page.

7. Built-in Effects and Animations

Provides ready-made effects like fade, slide, hide, and show.

8. Large Plugin Library

Thousands of plugins are available to extend functionality.

9. Open Source and Free

jQuery is freely available and widely supported by the developer community.

10. Improves Productivity

Faster development and easier maintenance increase developer efficiency.

jQuery Short Example

```
<p id="msg">Hello</p>
<button id="btn">Click</button>

<script>
$("#btn").click(function() {
    $("#msg").text("Hello jQuery");
});
</script>
```

(c) Write HTML and PHP program to read and store book information such as book id, title, author and price from database table. Output the data to a webpage in tabular format.

1) HTML Form (book.html)

(To enter and store book information)

```
<!DOCTYPE html>
<html>
<head>
    <title>Book Entry</title>
</head>
<body>

<h2>Add Book Information</h2>

<form method="post" action="book.php">
    Book ID: <input type="text" name="book_id"><br><br>
    Title: <input type="text" name="title"><br><br>
    Author: <input type="text" name="author"><br><br>
    Price: <input type="text" name="price"><br><br>
    <input type="submit" value="Save Book">
</form>

</body>
```

</html>

2) PHP Program (book.php)

(Insert data into database and display it in table format)

```
<?php
$conn = mysqli_connect("localhost", "root", "", "library");

if (!$conn) {
    die("Connection failed");
}

if (isset($_POST['book_id'])) {
    $id = $_POST['book_id'];
    $title = $_POST['title'];
    $author = $_POST['author'];
    $price = $_POST['price'];

    $insert = "INSERT INTO books VALUES
('{$id}', '{$title}', '{$author}', '{$price}')";
    mysqli_query($conn, $insert);
}

$result = mysqli_query($conn, "SELECT * FROM books");

echo "<h2>Book Information</h2>";
echo "<table border='1' cellpadding='10'>";
echo "<tr>
    <th>Book ID</th>
    <th>Title</th>
    <th>Author</th>
    <th>Price</th>
</tr>";

while ($row = mysqli_fetch_assoc($result)) {
    echo "<tr>";
    echo "<td>".$row['book_id']."</td>";
    echo "<td>".$row['title']."</td>";
    echo "<td>".$row['author']."</td>";
    echo "<td>".$row['price']."</td>";
    echo "</tr>";
}

echo "</table>";

mysqli_close($conn);
?>
```

3) Output

Book ID	Title	Author	Price
101	PHP Basics	R. Sharma	350
102	Web Programming	A. Patel	420

